SECUREBANK

Threat Modeling & Security Assessment Report

Application Security Portfolio Project

Scope: Flask demo API, JWT authentication, object-level authorization, audit logging, rate limiting, STRIDE analysis, regression tests, and CI security checks.

Repository	threat-modeling-lab
Verified commit	0bf880a — Test admin endpoint rate limiting
Local tests	35 passed in 2.70 seconds
CI evidence	GitHub Actions run #25: all four configured jobs succeeded

Assessment boundary

This report summarizes evidence recorded for the lab project. It is not an independent penetration test, production certification, or proof that all vulnerabilities have been eliminated. Claims are limited to the described implementation, tests, scans, and workflow run.

1. Executive Summary

SecureBank is a scoped demo API used to demonstrate threat modeling and practical application-security controls. The project combines STRIDE analysis with regression tests and CI security checks. The latest recorded local suite passed 35 tests, and GitHub Actions run #25 for commit 0bf880a showed successful results for Security Regression Tests, Semgrep SAST, Trivy Security Scan, and Gitleaks Secret Scan.

Key controls include JWT validation, account ownership checks, role-based protection for the admin health endpoint, structured audit logging with a hash chain, request rate limiting, and rejection of unsupported account mutation methods.

Overall assessment: controls are demonstrated in tested lab scenarios. Production-readiness items—including shared rate-limit storage, external audit-log protection, operational key management, and broader resource-exhaustion testing—remain open.

2. Scope and System Overview

The application is a Flask API with in-memory account data. The demo includes account IDs 101 and 202, JWT bearer authentication, a login endpoint, an account-read endpoint, and an admin health endpoint. The application requires a JWT signing secret through the environment and verifies password hashes for login.

In scope

- Authentication and JWT signature/claim validation.
- Object-level authorization for account access.
- Role authorization for the admin health endpoint.
- Audit events, persistence, permissions, and hash-chain verification.
- Rate limits on login, account, and admin health endpoints.
- HTTP method restrictions, automated tests, and configured CI checks.

Not established or outside current scope

- A production database and database query paths.
- Container image scanning in the current CI workflow.
- Production deployment architecture, external log storage, and key-management operations.
- Comprehensive distributed-client abuse and resource-exhaustion testing.

3. STRIDE Threat Summary

Threat	Controls and evidence	Status / residual risk
S-001 — Spoofing	JWT signature, expiry, issuer, and audience validation; negative token tests.	Partially validated. Broader spoofing controls and production key operations are not assessed.
T-001 — Tampering	Account ownership checks; POST, PUT, PATCH, and DELETE rejected with HTTP 405 on read-only account endpoint.	Partially validated. No mutation endpoint exists; future write protection remains unverified.
R-001 — Repudiation	Structured persistent JSONL audit events, restrictive permissions, and hash-chain verification.	Partially validated. A privileged attacker could rewrite logs and recompute hashes; external checkpoints and retention are not implemented.
I-001 — Information Disclosure	Server-side account ownership checks; audit filtering avoids logging selected sensitive fields.	Mitigated in tested scenarios; evidence covers current endpoints and test cases.
D-001 — Denial of Service	Flask-Limiter: 5 login requests/minute and 30 account/admin-health requests/minute per client IP; HTTP 429 tests.	Partially validated. Distributed-client abuse, broader exhaustion, and shared production storage remain unverified.
E-001 — Elevation of Privilege	Admin health endpoint requires authentication and admin role; tests cover denial, admin access, and missing token.	Partially validated. Persistent admin provisioning and future privileged operations are not established.

4. Key Security Controls and Test Evidence

Authentication and authorization

JWT checks include signature, expiry, issuer, audience, and required claims. The account endpoint checks object ownership, returning 403 for tested cross-account access. The admin health endpoint is guarded by authentication and a role check.

Audit logging

Audit events are stored as JSON Lines, use a SHA-256 hash chain, and apply restrictive directory/file permissions. Tests cover successful and failed login auditing, cross-account and missing-token events, persistence, chain verification, tampering detection, and file permissions. Hash chaining does not protect against a privileged attacker who can rewrite the entire log and recompute hashes.

Rate limiting

Login is limited to 5 requests/minute; account and admin-health endpoints are limited to 30 requests/minute per client IP. Tests cover login, account, and admin-health HTTP 429 behavior. Memory storage is suitable for the lab but is not shared across workers and resets on application restart.

Method restrictions

The account endpoint is read-only. Tests verify unsupported POST, PUT, PATCH, and DELETE requests return HTTP 405. Since no mutation endpoint exists, this does not establish the safety of future write operations.

5. Automated Validation

Check	Recorded evidence	Interpretation
Pytest	35 passed in 2.70 seconds locally	Regression tests passed for tested scenarios.
Semgrep SAST	GitHub Actions job succeeded	Configured static-analysis job passed; not proof of absence of all code flaws.
Trivy Security Scan	GitHub Actions job succeeded	Configured filesystem scan passed; this is not evidence of container image scanning.
Gitleaks Secret Scan	GitHub Actions job succeeded	Configured repository-history secret scan passed for this workflow run.

CI reference: commit 0bf880a, workflow run #25, “Test admin endpoint rate limiting.” The supplied screenshot shows Success and green checks for all four configured jobs.

6. Residual Risks and Recommended Follow-up

- **Rate limiting:** test distributed-client abuse and broader resource exhaustion; use shared storage in multi-worker production deployment.
- **Audit logging:** consider external/append-only storage, independent integrity checkpoints, and a defined retention policy.
- **Authentication:** document production key storage and rotation; validate deployment settings.
- **Authorization:** preserve object-level checks for each new object and operation; add tests for future privileged actions and mutation endpoints.
- **CI scanning:** tie results to commit hashes and review scanner output when workflow configuration or dependencies change.
- **Container images:** add image scanning if the project begins building or distributing images.
- **Database:** if introduced, add parameterized-query controls and database-specific injection tests.

7. Conclusion

SecureBank demonstrates a practical threat-modeling workflow: identify STRIDE threats, implement controls, write regression tests, record residual risks, and validate changes through CI. The latest recorded commit includes the admin rate-limit test and updates to the threat register, traceability matrix, security test plan, and README. Present it as a lab portfolio demonstration with explicit limitations—not as production certification or an exhaustive penetration-test report.

Evidence snapshot: commit 0bf880a · GitHub Actions run #25 · local pytest 35 passed in 2.70s · working tree previously verified clean.